# TRENTOS-M SysLogger

**Page Content**



# Overview

The SysLogger component enqueues log messages from its clients and makes them available to backend spoolers that know how to process them. For example, a UART spooler that has access to the serial port and outputs the messages there. A file spooler will write the message to a log file.
The SysLogger is agnostic of the backend implementations. Every spooler has its own queues with its own capability in terms of maximum amount of items. Spoolers may be blocking or non-blocking (at the cost of data loss). Spoolers may also have different queue sizes. A queue can be configured with a filter mask that decided which log messages enter the queue and which ones are dropped by the queue.

In general the requirements for such a system logger are the following:

- a CAmkES interface consisting of
  - an RPC interface

    ```
    OS_Error_t log(in unsigned int level, in string message);
    ```

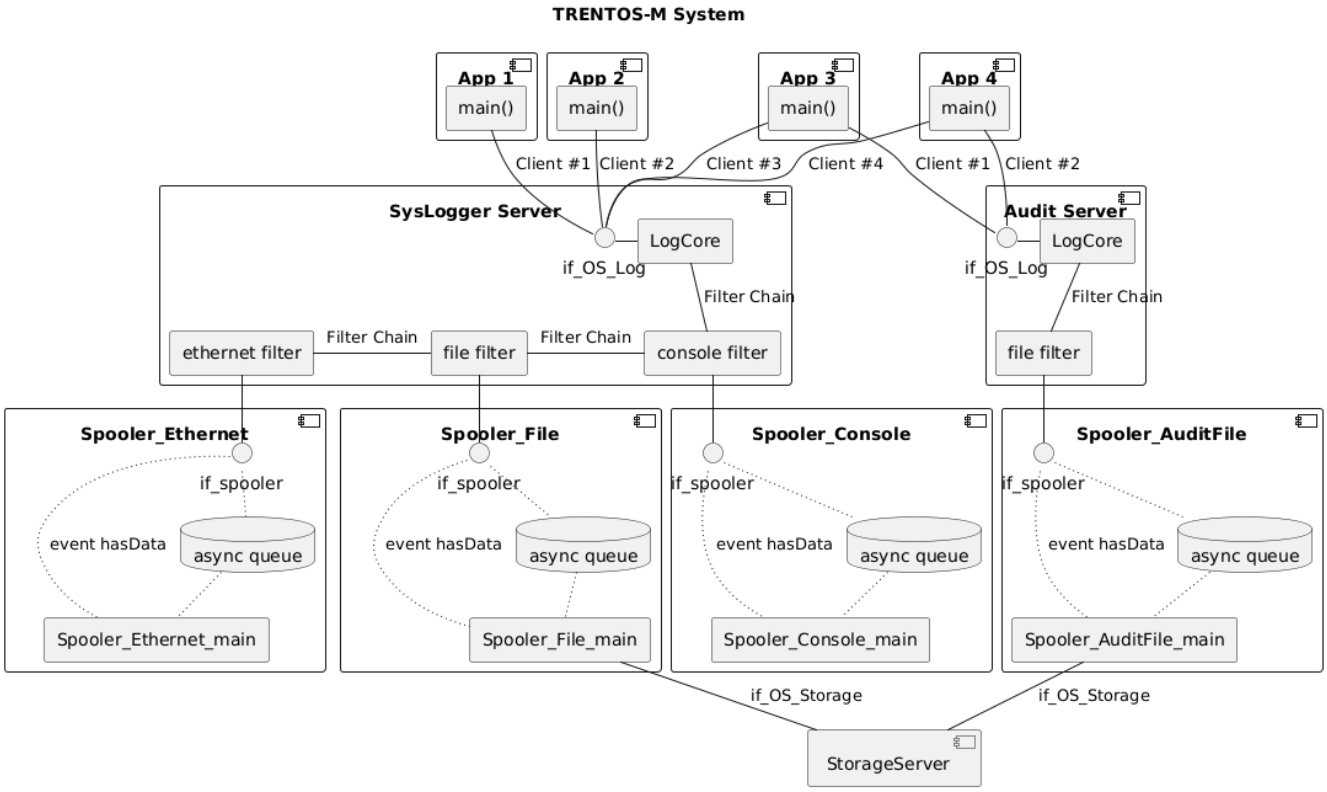
  - a dataport for the messages
  - an event for
- helper macros facility to declate and connect the interfaces
- a system logger and at lesat one spooler implementation
  - logging queue chain implementation (the behavior of propagation of the log message thru the different queues. One per each spooler backend connected)
  - SysLogger shares the logs with the spoolers by granting access to their dedicated FIFOs (via FifoDataport data structures)
  - A dedicated logging queue accepts a log if the level matches its logging queue mask and will call the enqueue() like function of its next linked logging queue
  - spoolers get notified when new entries are available in the FIFO dataport
  - no logger chain (therefore no spooler attached) means,  printf() serializer behavior. This special requirement is for the simplest configurations were the need there is just to not interrupt the printf() called from different components and there is no need to make the log() function being as less blocking as possible, which is instead the required behavior when logging is required to be as less as possible invasive in terms of timing alteration.

## Architecture

### First Sketch

blocked URL

### Full flexible Vision




# Implementation

## SysLoggerClient

Usage concept:

- SysLoggerClient needs to be initialized and freed using **SysLoggerClient_init()** and **SysLoggerClient_free()** functions
- A client component may use directly the SysLoggerClient function **SysLoggerClient_log()** or it may provide that log function as backend of Debug.h module, so that every **Debug_LOG_*** macro redirects the log message to the SysLogger component. This is very easy and requires your client component to just include the syslogger_client library when getting declare in CMakeLists.txt file. For example:

**SysLoggerClient - Usage**

```
    DeclareCAmkESComponent(
    NwApp1
    SOURCES
        components/TestAppClient/test_app_client.c
    C_FLAGS
        -Wall
        -Werror
    LIBS
        system_config
        os_core_api
        lib_debug
        lib_macros
        os_network_api
        syslogger_client
)
```

## Component

**SysLogger - Overview**

```
    // Definition and declaration of component -----------------------------------
/*
 * To be used when the SysLogger is not connect to any spooler but rather it
 * does process the logs internally (e.g. acting as printf serializer)
 */
#define SysLogger_COMPONENT_DEFINE_NO_SPOOLERS(\
    _name_\
)\
component _name_ {\
    SysLogger_DECLARE_BODY()\
}

/*
 *   To be used when the SysLogger is connected to spoolers
 *
 *      SysLogger_COMPONENT_DEFINE(
 *          <name>,
 *          <spooler_instance>, <buf_size>,
 *          <spooler_instance>, <buf_size>,
 *          ...
 *      )
 */
#define SysLogger_COMPONENT_DEFINE(\
    _name_,\
    ...\
)\
component _name_ {\
    SysLogger_DECLARE_BODY()\
\
    SysLogger_DECLARE_SPOOLER_CONNECTORS(__VA_ARGS__)\
}

/*
 *   To be used when the SysLogger is connected to spoolers
 *
 *      SysLogger_INSTANCE_DECLARE(
 *          <name>, <inst>,
 *          <spooler_instance>,
 *          <spooler_instance>,
 *          ...
 *      )
 */
#define SysLogger_INSTANCE_DECLARE(\
    _name_,\
    _inst_,\
    ...\
)\
    component _name_  _inst_;\
    SysLogger_INSTANCE_CONNECT_SPOOLERS(_inst_, __VA_ARGS__)

// Backend connections -------------------------------------------------------
/*
 *      SysLogger_INSTANCE_CONNECT_SPOOLERS(
 *          <inst>,
 *          <spooler_instance>,
 *          <spooler_instance>,
 *          ...
 *      )
 */
#define SysLogger_INSTANCE_CONNECT_SPOOLERS(\
    _syslogger_inst_,\
    ...\
)\
    FOR_EACH_0P(SysLogger_INSTANCE_CONNECT_SPOOLER,\
                _syslogger_inst_,\
                __VA_ARGS__)

// Client connections --------------------------------------------------------
/*
 *      SysLogger_INSTANCE_CONNECT_CLIENTS(
```

```
 *          <inst>,
 *          <client_instance>,
 *          <client_instance>,
 *          ...
 *      )
 */
#define SysLogger_INSTANCE_CONNECT_CLIENTS(\
    _syslogger_inst_,\
    ...\
)\
    FOR_EACH_0P(SysLogger_INSTANCE_CONNECT_CLIENT,\
                _syslogger_inst_,\
                __VA_ARGS__)
```

### Interface

**SysLogger - Interface**

```
    procedure if_OS_Log
{
    include "OS_Error.h";
    OS_Error_t log(in unsigned int level, in string message);
};
```

# Usage

This is how the component can be instantiated in the system.

## Declaration of the Component in CMake

**SysLogger - Usage 1**

```
    DeclareCAmkESComponent_SysLogger(
    SysLogger
    system_config
)
```

## Instantiation and Configuration in CAmkES

### Defining the Component

When the SysLogger component is defined, the spooler to which it is expect to be connected have to be set up as well by providing the names of their instances and the size for the shared FIFO dataport:

**SysLogger - Usage 2**

```
    #include "SysLogger/camkes/SysLogger.camkes"

SysLogger_COMPONENT_DEFINE(
        <name>,
        <spooler_instance>, <buf_size>,
        <spooler_instance>, <buf_size>,
        ...
)
```

Othewise it is possible to define a SysLogger component without spoolers (printf serializer):

**SysLogger - Usage 3**

```
    #include "SysLogger/camkes/SysLogger.camkes"

SysLogger_COMPONENT_DEFINE_NO_SPOOLERS(
        <name>
)
```

### Declaring the Component

When the SysLogger is declared, the spoolers to which it is connected have to be set up as well:

**SysLogger - Usage 4**

```
    #include "SysLogger/camkes/SysLogger.camkes"

SysLogger_INSTANCE_DECLARE(
        <name>, <inst>,
        <spooler_instance>,
        <spooler_instance>,
        ...
)
```

### Connecting the clients

**SysLogger - Usage 5**

```
    SysLogger_INSTANCE_CONNECT_CLIENTS(
        <inst>,
        <client_instance>,
        <client_instance>,
        ...
)
```

# Example

## Configuration of the Component in system_config.h

**SysLogger - Example 1**

```
    #define SysLogger_Config_MSG_SIZE               SysLogger_MAX_MSG_SIZE // size of messages
#define SysLogger_Config_COMPONENT_NAME_COLUMNS  11                                                 // columns
reserved to print the component name before the message itself
```

## Instantiation and Configuration in CAmkES

Note that the component must be defined first. SysLogger cannot come with a predefined component as the spooler components cannot be determined beforehand.

**SysLogger - Example 4**

```
    #include "SysLogger/camkes/SysLogger.camkes"
SysLogger_COMPONENT_DEFINE_NO_SPOOLERS(SysLogger)

component SysLogger sysLogger;

SysLogger_INSTANCE_CONNECT_CLIENTS(sysLogger,
        nwStack1,
        nwStack2,
        nwApp1,
        nwApp1_2,
        nwApp2
)
```